Lab 10 – Binary Search Tree

Task03:

You're a manager managing a **warehouse inventory system** where you organize item **prices (in dollars)** into a **binary search tree (BST)**. The system helps you efficiently manage items by categorizing prices **above the average ($13.73)** on the **right side** and prices **below the average** on the **left side** of the tree giving you powerful tools to analyze and update your inventory seamlessly.

11.0	13.5	9.5	12.5	14.0	15.5	8.0	10.5	11.5	13.0	15.5	18.0	17.5	19.0	17.0

Write a program in C++ and perform following task:

a. Display prices in sorted order
b. Display the number of items priced above the average.
c. Display the number of items priced below the average.
d. Display the highest price.
e. Display the lowest price.
f. Search for a price provided by the user.
g. Delete a price provided by the user.

Code:

```
#include<iostream>
#include <queue>
using namespace std;

class Node{
public:
    float data;
    bool isDummy;
    Node *left, *right;

    Node(float data, bool isDummy = false){
        this->data = data;
        this->isDummy = isDummy;
        left = right = nullptr;
    }
};

// a. display prices in sorted order
void displaySortedPrices(Node* root){
    if(!root) return;
```

```
    displaySortedPrices(root->left);
    cout << root->data << " ";
    displaySortedPrices(root->right);
}

// b. display no of prices below average
void diplayPricesBelowAvg(Node* root){
    if(root->left == nullptr){ // empty check
        cout << "There is no price below average in the tree.\n";
    }

    Node* temp = root->left;
    int count = 0;
    queue<Node*> q;
    q.push(temp);

    // doing simple level order traversal
    while(!q.empty()){
        int size = q.size();

        for(int i = 0; i < size; i++){
            Node* curr = q.front();
            q.pop();
            count++;

            if(curr->left != nullptr) q.push(curr->left);
            if(curr->right != nullptr) q.push(curr->right);
        }
    }
    cout << "Items below average: " << count << ".\n";
}

// c. no of prices above average

// d. diaplay the highest price
int getHighestPrice(Node* root){
    if(root->right != nullptr){
        Node* temp = root->right;

        while(temp->right != nullptr) temp = temp->right;

        return temp->data;
    }
    else if(root->left != nullptr){
        Node* temp = root->left;
```

```
        while(temp->right != nullptr) temp = temp->right;
        return temp->data;
    }

    return -1;
}

int main(){
    Node* root = new Node(13.273, true); // anchor value to split data


}
```